

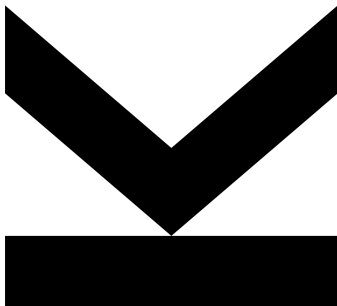
Sophie Öttl
k12308187,
Simon Grundner
k12136610

Institut für Integrierte
Schaltungen

 <https://jku.at/iic>

5. Februar 2026

Protokoll



Digitale Schaltungstechnik PR - WS2025



JOHANNES KEPLER
UNIVERSITÄT LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Inhaltsverzeichnis

1. Ampelsteuerung	5
1.1 Vorbereitung	5
2. Reaktionsspiel	6
2.1 Vorbereitung	6
2.1.1 Widerstandsberechnung	6
2.1.2 Fragen	6
2.2 Durchführung	6
3. Realisierung eines Logic Analyzers	7
3.1 Vorbereitung	7
3.2 Durchführung	7
3.3 Ergebnisse	7
4. Wertebestimmung von Widerstand und Kondensator	8
4.1 Vorbereitung	8
4.1.1 Widerstandsbestimmung	8
4.1.2 Spannungsteiler	8
4.1.3 Kondensatorladung	9
4.2 Durchführung	9
4.3 Ergebnisse	10
4.3.1 Widerstand 1	10
4.3.2 Widerstand 2	11
4.3.3 Widerstand 3	12
4.3.4 Widerstand 4	13
4.3.5 Kondensator 1	14
4.3.6 Kondensator 2	14
4.3.7 Kondensator 3	14
5. Ansteuerung eines Lautsprechers	15
5.1 Vorbereitung	15
5.1.1 2-Bit R2R Netzwerk	15
5.1.2 Sinus Wellentabelle	16
5.2 Durchführung	17
5.2.1 Aufbau am Steckbrett	17
5.2.2 Programmierung	17
5.3 Ergebnisse	17
6. Motorsteuerung mit Näherungssensor	18
6.1 Vorbereitung	18
6.1.1 Ultraschall-Modul HC-SR04 information	18
6.1.2 Plan für die Verwendung des Ultraschall Moduls	18
6.1.3 Schieberegister für 7 segment Anzeige	18
6.1.4 Berechnung Vorwiderstände	18
6.2 Praktikumsaufgabe	20
6.2.1 e	20
7. Datenübertragung mittels Infra-Rot (IR)	21
7.1 Vorbereitung	21
7.1.1 Morse Code Symbole	21

7.1.2	Codierung des Morsesignals	21
7.1.3	Widerstandsberechnung	22
7.2	Durchführung	22
7.2.1	Aufbau am Steckbrett	22
7.2.2	Implementierung des Senders	23
7.2.3	Implementierung des Empfängers	24
7.3	Ergebnisse	25

Geräteliste

Tabelle 1: Caption

Pos.	Bezeichnung	Serien Nummer
1	Multimeter	IH807686
2	Oszilloskop	

1. Ampelsteuerung

1.1 Vorbereitung

Im Betrieb weisen die verwendeten Leuchtdioden eine Vorwärtsspannung von ca. $U_F = 2\text{ V}$ auf. Um den Strom durch die Leuchtdiode zu begrenzen, wird ein Vorwiderstand so dimensioniert, dass die restliche Spannung mit dem gewählten Maximalstrom an ihm abfällt.

$$R_V = \frac{U_{IO} - U_F}{I_D} = \frac{5\text{ V} - 2\text{ V}}{15\text{ mA}} = 200\ \Omega$$

Zur Verfügung steht die E6 Widerstandsbaureihe. Damit keinesfalls der Maximalstrom überschritten wird, wird der nächst höhere Wert aus der Reihe gewählt.

$$R_V = 220\ \Omega$$

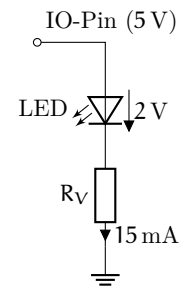


Abbildung 1: LED mit Vorwiderstand

2. Reaktionsspiel

2.1 Vorbereitung

2.1.1 Widerstandsberechnung

Berechnet:

$$R_{V,calc} = \frac{5\text{ V} - 2\text{ V}}{15\text{ mA}} = 200\ \Omega$$

Aus E6-Reihe gewählt:

$$R_V = 220\ \Omega$$

2.1.2 Fragen

? Wie kann das Mikrocontroller-Programm zu einem beliebigen Zeitpunkt gestoppt werden?
Wie kann ausgewertet werden, ob der Taster zum richtigen oder zum falschen Zeitpunkt gedrückt wurde?

Das Programm kann mittels eines **externen Interrupts** zu einem beliebigen Zeitpunkt unterbrochen werden. Ein Interrupt wird dazu an einen Pin konfiguriert.

Sobald der Interrupt ausgelöst wird, springt das Programm direkt in die **Interrupt-Service-Routine (ISR)**. Dort werden Variablen gesetzt, welche die `loop`-Methode anschließend überprüft, um den Ablauf zu stoppen.

Zudem kann in der ISR der aktuelle Status (z.B. welche LED gerade aktiv ist) gespeichert werden. Da der Interrupt sofort reagiert, lässt sich so exakt auswerten, ob der Taster zum richtigen Zeitpunkt gedrückt wurde.

2.2 Durchführung

- Tinker CAD Link
- Stückliste

```
1 const int ledPins[] = {3, 4, 5, 6, 7};
2 const int buttonPin = 2;
3
4 for (int i = 0; i < 5; i++) {
5     pinMode(ledPins[i], OUTPUT);
6 }
7 pinMode(buttonPin, INPUT_PULLUP);
8 attachInterrupt(digitalPinToInterrupt(buttonPin), handlePress, FALLING);
```

Da nur die Pins 2 und 3 mit einem Interrupt konfiguriert werden können, wurde der Taster mit Pin 2 verbunden. Pin 3 bis 7 sind von den LEDs belegt.

3. Realisierung eines Logic Analyzers

3.1 Vorbereitung

AND		
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

$$Y = A \wedge B$$

OR		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

$$Y = A \vee B$$

NAND		
A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

$$Y = \overline{A \wedge B}$$

NOR		
A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

$$y = \overline{A \vee B}$$

XOR		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$Y = A \oplus B$$

3.2 Durchführung

- Tinker CAD Link
- Stückliste

3.3 Ergebnisse

Gatter 1	Gatter 2	Gatter 3
OR	NAND	XOR

4. Wertebestimmung von Widerstand und Kondensator

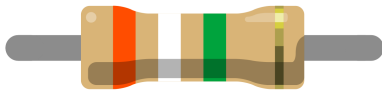
4.1 Vorbereitung

4.1.1 Widerstandsbestimmung



Widerstandswert: 47 kΩ

Toleranz: ±5 %



Widerstandswert: 3.9 MΩ

Toleranz: ±5 %



Widerstandswert: 6.8 kΩ

Toleranz: ±10 %

4.1.2 Spannungsteiler

- a) Die Spannung teilt sich am Spannungsteiler im Verhältnis der Widerstände auf.

$$U_{R1} = U_e - U_a = 5 \text{ V} - 3 \text{ V} = 2 \text{ V}$$

Das Verhältnis der Widerstände $R_1 : R_2$ entspricht dem Verhältnis der Spannungen $U_{R1} : U_a$, also **2:3**. Berechnung von R_2 mit $R_1 = 10 \text{ kΩ}$:

$$\begin{aligned} \frac{R_1}{R_2} &= \frac{U_{R1}}{U_a} \\ R_2 &= R_1 \cdot \frac{U_a}{U_{R1}} \\ R_2 &= 10 \text{ kΩ} \cdot \frac{3 \text{ V}}{2 \text{ V}} \\ R_2 &= 15 \text{ kΩ} \end{aligned}$$

- b) Der Widerstand R_2 wird durch R_x ersetzt. Die neue Ausgangsspannung beträgt $U_a = 1 \text{ V}$.

$$U_{R1} = 5 \text{ V} - 1 \text{ V} = 4 \text{ V}$$

Berechnung von R_x :

$$\begin{aligned} R_x &= R_1 \cdot \frac{U_a}{U_{R1}} \\ R_x &= 10 \text{ kΩ} \cdot \frac{1 \text{ V}}{4 \text{ V}} \\ R_x &= 2.5 \text{ kΩ} \end{aligned}$$

Abbildung 2: Messschaltung zur Strom und Spannungsmessung eines Widerstands.

4.1.3 Kondensatorladung

a) Die Spannung am Kondensator folgt der Ladefunktion:

$$u_c(t) = U_q \cdot \left(1 - e^{-\frac{t}{\tau}}\right)$$

Es ist bekannt, dass der Kondensator nach der Zeitkonstante $\tau = R \cdot C$ circa 63 % der Endspannung erreicht hat ($1 - e^{-1} \approx 0.63$). Da nach $t = 100 \text{ ms}$ genau dieser Wert erreicht ist, gilt:

$$\tau = 100 \text{ ms}$$

Daraus lässt sich die Kapazität C berechnen:

$$\tau = R \cdot C$$

$$C = \frac{\tau}{R}$$

$$C = \frac{0.1 \text{ s}}{10 \text{ k}\Omega}$$

$$C = 0.00001 \text{ F} = 10 \text{ }\mu\text{F}$$

b) Der Kondensator gilt als vollständig geladen (Spannung $> 99\%$), wenn eine Zeit von ca. $5 \cdot \tau$ vergangen ist ($1 - e^{-5} \approx 0.993$).

Die Ladedauer beträgt somit:

$$t_{\text{voll}} \approx 5 \cdot \tau$$

$$t_{\text{voll}} = 5 \cdot 100 \text{ ms}$$

$$t_{\text{voll}} = 500 \text{ ms}$$

4.2 Durchführung

- Tinker CAD Link Aufgabe 1
- Tinker CAD Link Aufgabe 2
- Stückliste

Bilder

- Schaltplan (tinkercad)
- Strom und Spannungsmessung

4.3 Ergebnisse

4.3.1 Widerstand 1

Ohmmeter gemessen: 100.3Ω

Farbcode Der Widerstand weist 5 Ringe mit folgenden Farben auf:

Braun	Schwarz	Schwarz	Schwarz	Braun
-------	---------	---------	---------	-------

Diese Farbkombination entspricht $100 \Omega \pm 1 \%$.

UI-Messung Messung von Strom und Spannung durch den Widerstand.

Tabelle 2: Strom und Spannungsmessung gemäß Abbildung 2 mit anschließender Berechnung des Widerstandes mit dem Ohmschen Gesetz.

Strom / A	Spannung / V	Widerstand / Ω
0.500	0.0505	

Programm Vom Programm wurden folgende Werte über die serielle Schnittstelle empfangen:

Tabelle 3: Ausgabe des berechneten Widerstandswertes und dem ADC Messwert.

Widerstandswert / Ω	ADC Messwert
78.50	8
58.76	6
68.62	7
78.50	8

4.3.2 Widerstand 2

Ohmmeter Gemessen: 12.01 k Ω

Farbcode Der Widerstand weist 4 Ringe mit folgenden Farben auf:

Braun	Rot	Orange	Gold
-------	-----	--------	------

Diese Farbkombination entspricht 12 k $\Omega \pm 5\%$.

UI-Messung Messung von Strom und Spannung durch den Widerstand.

Tabelle 4: Strom und Spannungsmessung gemäß Abbildung 2 mit anschließender Berechnung des Widerstandes mit dem Ohmschen Gesetz.

Strom / A	Spannung / V	Widerstand / Ω
0.231	2.759	

Programm Vom Programm wurden folgende Werte über die serielle Schnittstelle empfangen:

Tabelle 5: Ausgabe des berechneten Widerstandswertes und dem ADC Messwert.

Widerstandswert / Ω	ADC Messwert
11891.41	557
11891.41	557
11891.41	557
11891.41	557

4.3.3 Widerstand 3

Ohmmeter gemessen: 185.7 k Ω

Farbcode Der Widerstand weist 5 Ringe mit folgenden Farben auf:

Braun	Grau	Schwarz	Orange	Braun
-------	------	---------	--------	-------

Diese Farbkombination entspricht 180 k $\Omega \pm 1\%$.

UI-Messung Messung von Strom und Spannung durch den Widerstand.

Tabelle 6: Strom und Spannungsmessung gemäß Abbildung 2 mit anschließender Berechnung des Widerstandes mit dem Ohmschen Gesetz.

Strom / mA	Spannung / V	Widerstand / Ω
0.026	4.8	

Programm Vom Programm wurden folgende Werte über die serielle Schnittstelle empfangen:

Tabelle 7: Ausgabe des berechneten Widerstandswertes und dem ADC Messwert.

Widerstandswert / Ω	ADC Messwert
179090.73	970
186362.31	972
182657.92	971
194215.59	974

4.3.4 Widerstand 4

Ohmmeter gemessen: 1.01 k Ω

Farbcode Der Widerstand weist 5 Ringe mit folgenden Farben auf:

Braun	Schwarz	Schwarz	Braun	Braun
-------	---------	---------	-------	-------

Diese Farbkombination entspricht 1 k $\Omega \pm 1\%$.

UI-Messung Messung von Strom und Spannung durch den Widerstand.

Tabelle 8: Strom und Spannungsmessung gemäß Abbildung 2 mit anschließender Berechnung des Widerstandes mit dem Ohmschen Gesetz.

Strom / A	Spannung / V	Widerstand / Ω
0.459	0.464	

Programm Vom Programm wurden folgende Werte über die serielle Schnittstelle empfangen:

Tabelle 9: Ausgabe des berechneten Widerstandswertes und dem ADC Messwert.

Widerstandswert / Ω	ADC Messwert
960.71	90
972.42	91
960.71	90
960.71	90

4.3.5 Kondensator 1

Marking code Am Bauteil wurde der Wert 220 μF abgelesen.

Programm Vom Programm wurden folgende Werte über die Serielle Schnittstelle empfangen:

- Ladezeit: 2 397 484 μs
- Kapazität: 239.75 μF

4.3.6 Kondensator 2

Marking code Am Bauteil wurde der Wert 47 μF abgelesen.

Programm Vom Programm wurden folgende Werte über die Serielle Schnittstelle empfangen:

- Ladezeit: 449 468 μs
- Kapazität: 44.95 μF

4.3.7 Kondensator 3

Marking code Am Bauteil wurde der Wert 10 μF abgelesen.

Programm Vom Programm wurden folgende Werte über die serielle Schnittstelle empfangen:

- Ladezeit: 103 276 μs
- Kapazität: 10.33 μF

5. Ansteuerung eines Lautsprechers

5.1 Vorbereitung

5.1.1 2-Bit R2R Netzwerk

Ein 2-Bit Digital-Analog Wandler wird mit einem R2R-Netzwerk realisiert.

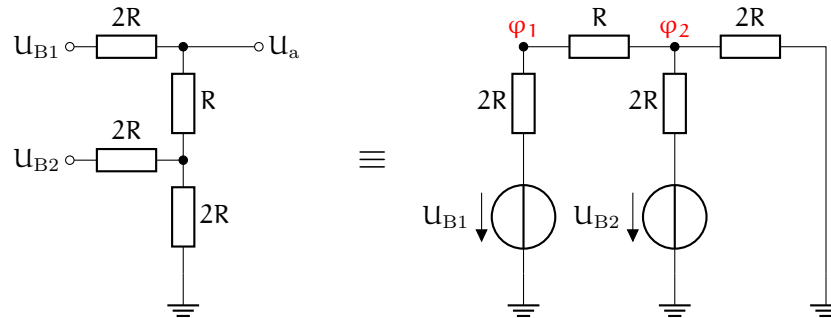


Abbildung 3: R2R Netzwerk

Um das Netzwerk zu lösen, wird das Knotenpotential-Verfahren verwendet. Das Netzwerk weist dabei zwei Knoten mit den Potenzialen φ_1 und φ_2 bezüglich Masse auf, wobei zu erkennen ist, dass $\varphi_1 = U_a$ ist. Es lässt sich ein Gleichungssystem der Form

$$\begin{bmatrix} G_{11} & -G_{12} \\ -G_{21} & G_{22} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} = \begin{bmatrix} I_1 \\ I_2 \end{bmatrix} \quad (1)$$

aufstellen. Dabei ist G_{ii} die Summe der an Knoten i anliegenden Leitwerte und G_{ij} die Summe der Leitwerte, welche Knoten i und Knoten j verbinden. Es gilt $G_{ij} = G_{ji}$ wenn $i \neq j$. I_i sind die zum Knoten i zufließenden Quellströme, diese können aus den realen Spannungsquellen mit der jeweiligen Ersatzstromquelle berechnet werden.

$$\begin{aligned} \begin{bmatrix} \frac{1}{2R} + \frac{1}{R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{1}{R} + \frac{1}{2R} + \frac{1}{2R} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \frac{1}{2R} \\ U_{B2} \frac{1}{2R} \end{bmatrix} \\ 2R \begin{bmatrix} \frac{3}{2R} & -\frac{1}{R} \\ -\frac{1}{R} & \frac{4}{2R} \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \\ U_{B2} \end{bmatrix} \\ 2I + II \rightarrow \begin{bmatrix} 3 & -2 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} U_{B1} \\ U_{B2} \end{bmatrix} \\ \begin{bmatrix} 4 & 0 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} \varphi_1 \\ \varphi_2 \end{bmatrix} &= \begin{bmatrix} 2U_{B1} + U_{B2} \\ U_{B2} \end{bmatrix} \end{aligned}$$

Aus der ersten Zeile des Gleichungssystems und mit $\varphi_1 = U_a$ folgt nun

$$4\varphi_1 = 2U_{B1} + U_{B2} \implies U_a = \frac{U_{B1}}{2} + \frac{U_{B2}}{4} \quad (2)$$

Die Ausgangswerte bei verschiedenen Eingangsstellungen sind in Tabelle 10 gezeigt. Das Muster in Gleichung 2 lässt sich auch für 4-Bit fortsetzen

$$U_{a,4\text{-Bit}} = \frac{U_{B1}}{2} + \frac{U_{B2}}{4} + \frac{U_{B3}}{8} + \frac{U_{B4}}{16}$$

Tabelle 10: Ausgänge des 2-Bit R2R Netzwerkes bei verschiedenen Eingangsstellungen

U_{B1}	U_{B2}	U_a
0 V	0 V	0 V
0 V	U_{Bit}	$\frac{1}{4} U_{Bit}$
U_{Bit}	0 V	$\frac{1}{2} U_{Bit}$
U_{Bit}	U_{Bit}	$\frac{3}{4} U_{Bit}$

5.1.2 Sinus Wellentabelle

a) Die Amplitude eines Sinus wird über eine Periode in 20 Werte diskretisiert. Die diskreten Werte erhält durch lineares Abbilden des Sinus auf den 4 bit Wertebereich.

$$D_{SIN} = \text{round} \left(U_{SIN} \frac{D_{MAX}}{U_{MAX}} \right) = \text{round} \left(U_{SIN} \frac{15}{5 \text{ V}} \right)$$

Die Berechneten Werte sind in Tabelle 11 aufgeführt.

Tabelle 11: Amplituden des Sinus im Bereich 0-5 V, unterteilt in 20 Stützwerte.

U_{SIN} / V	D_{SIN} Dezimal	D_{SIN} Hexadezimal
2.5	8	0x8
3.3	10	0xA
4.0	12	0xC
4.5	14	0xE
4.9	15	0xF
5.0	15	0xF
4.9	15	0xF
4.5	14	0xE
4.0	12	0xC
3.3	10	0xA
2.5	8	0x8
1.7	5	0x5
1.0	3	0x3
0.5	2	0x2
0.1	0	0x0
0.0	0	0x0
0.1	0	0x0
0.5	2	0x2
1.0	3	0x3
1.7	5	0x5

b) In einer Periodendauer T müssen alle $N = 20$ Werte der Wellentabelle durchlaufen werden. Die Zeit, wie lange ein Bitmuster am Ausgang anliegen muss, ergibt sich durch

$$t_b = \frac{T}{N} = \frac{1}{fN}.$$

Die berechneten Werte sind in Tabelle 12 dargestellt.

Tabelle 12: Caption

Ton	Frequenz f / Hz	Zeit pro Bitmuster t_b / μs
c'	262	190.84
d'	294	170.07
e'	330	151.52
f'	349	143.27
g'	392	127.55
a'	440	113.64
h'	494	101.21

5.2 Durchführung

- Tinker CAD Link
- Stückliste

5.2.1 Aufbau am Steckbrett

5.2.2 Programmierung

5.3 Ergebnisse

6. Motorsteuerung mit Näherungssensor

- Tinker CAD Link
- Stückliste

6.1 Vorbereitung

6.1.1 Ultraschall-Modul HC-SR04 information

Folgende wichtige Spezifikationsdaten konnten auf Mikrocontroller.net gefunden werden:

- Messbereich: 2 cm bis ca. 300 cm.
- Auflösung: Systembedingt ca. 3 mm.
- Versorgung: 5V Spannung bei einer Stromaufnahme von weniger als 2 mA.
- Messrate: Maximal 50 Messungen pro Sekunde (Intervall von 20 ms).

Pin Belegung (Von links nach Rechts):

- **Pin 1:** VCC (5 V)
- **Pin 2:** Trigger eingang TTL (0-0.8V ist low, 2,4-5V ist High)
- **Pin 3:** Ausgang Messergebnis TTL
- **Pin 4:** Ground

Eine Messung wird gestartet durch eine fallende Flanke am Triggereingang (Pin 1) die 10 μ sec andauert.

Das modul sendet nach 250 μ sec ein signal. Nun geht der ausgang auf High. Sobald das Echo empfangen wird geht der ausgang wieder auf low. Wird kein echo empfangen, bleibt der ausgang für 200ms auf High. Damit ergibt sich die entfernung als:

$$d = \frac{t \cdot 34.3 \frac{\text{cm}}{\text{ms}}}{2}$$

Das Ergebnis kann durch Temperatur, rauhe oberflächen und schlechtem winkel verschlechtert werden.

6.1.2 Plan für die Verwendung des Ultraschall Moduls

Der Grobe Plan für die verwendung ist, dass der Trigger und der Ausgangs Pin mit dem Arduino verbunden werden. Es werden Interrupts für den ausgang konfiguriert. Bei steigender Flanke wird die aktuelle systemzeit gespeichert. Bei fallender Flanke wird mit der differenz der aktuellen zeit und der gespeicherten zeit die Distanz berechnet und ein boolean gesetzt um anzuzeigen, dass die messung bereit ist,

Im hauptprogramm wird der steuer pin kurz auf high und wieder auf low gesetzt um eine messung zu triggern. Anschließend wird ein delay von 30ms durchgeführt damit wir die maximale Messrate nicht unterschreiten. Anschließend wird falls der Mess-boolean gesetzt wurde der Motor Ausgeschaltet.

6.1.3 Schieberegister für 7 segment Anzeige

6.1.4 Berechnung Vorwiderstände

Die Benötigte Dioden Spannung der LEDs ist $U_D = 2 \text{ V}$ bei einem Strom $I_D = 15 \text{ mA}$. Es ist mit einer Spannung von $U_{CC} = 5 \text{ V}$ an den Ausgängen des Schieberegisters zu rechnen. Der Vorwiderstand Kann wie folgt berechnet werden:

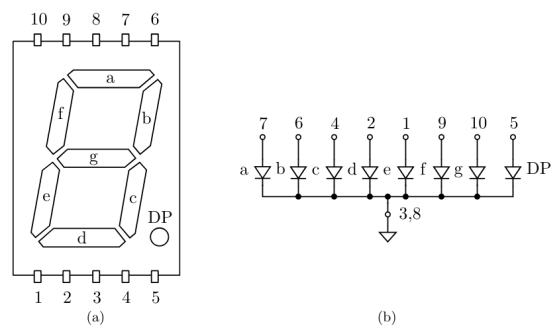


Abbildung 4: 7-Segment Anzeige

Ziffer	a	b	c	d	e	f	g	dp
0	1	1	1	1	1	1	0	0
1	0	1	1	0	0	0	0	0
2	1	1	0	1	1	0	1	0
3	1	1	1	1	0	0	1	0
4	0	1	1	0	0	1	1	0
5	1	0	1	1	0	1	1	0
6	1	0	1	1	1	1	1	0
7	1	1	1	0	0	0	0	0
8	1	1	1	1	1	1	1	0
9	1	1	1	1	0	1	1	0

Tabelle 13: Segmentbelegung für 7-Segment-Anzeige (High-aktiv)

$$U_{R_V} = U_{CC} - U_D = 5\text{ V} - 2\text{ V} = 3\text{ V}$$

$$I_{R_V} = I_D = 15\text{ mA}$$

$$R_V = \frac{U_{R_V}}{I_{R_V}} = \frac{3\text{ V}}{15\text{ mA}} = 200\ \Omega$$

Die E6 Reihe enthält die Werte (1,0 1,5 2,2 3,3 4,7 6,8). Damit sind die nächsten 2 Widerstände $150\ \Omega$ und $220\ \Omega$. Da der Strom von $I_D = 15\text{ mA}$ nicht überschritten werden soll, wählen wir den größeren Widerstand. Somit ist der Vorwiderstand: $R_V = 220\ \Omega$

6.2 Praktikumsaufgabe

6.2.1 e

7. Datenübertragung mittels Infra-Rot (IR)

Mit zwei Arduino Uno Boards soll eine Übertragung von Morsecodes mittels Infra-Rot Lichtsignalen aufgebaut werden. Sender seitig wird eine IR-LED angesteuert, sodass Infrarotsignale mit einer Frequenz von 38 kHz gesendet werden. Die Symbole der Morsezeichen (–, •) werden dabei mit unterschiedlicher Sendedauer enkodiert. Als Empfänger wird das IR-Detektor-Modul KY-022 verwendet, welches am Ausgang LOW ausgibt, wenn es ein IR-Lichtsignal mit der genannten Frequenz detektiert, ansonsten HIGH.

7.1 Vorbereitung

7.1.1 Morse Code Symbole

Eine Liste der Morse-Codewörter für das Alphabet ist in Tabelle 14 aufgeführt.

Tabelle 14: Morsealphabet

Buchstabe	Code	Buchstabe	Code	Buchstabe	Code
A	• –	J	• – – –	S	• • •
B	– • • •	K	– • –	T	–
C	– • – •	L	• – • •	U	• • –
D	– • •	M	– –	V	• • • –
E	•	N	– •	W	• – –
F	• • – •	O	– – –	X	– • • –
G	– – •	P	• – – •	Y	– • – –
H	• • • •	Q	– – • –	Z	– – • •
I	• •	R	• – •		

7.1.2 Codierung des Morsesignals

Um die Symbole der Morse-Codewörter zu übertragen, wird mit der Funktion `tone(pin, frequency, duration)` ein 38 kHz Rechtecksignal mit unterschiedlicher Dauer auf dem LED-GPIO Pin ausgegeben.

Algorithmus 1 Ablauf zum Senden der Korrekten Symbolauern in einem Morse-Codewort

```

for all Symbols in Codeword do
    tone(TX_PIN, F_IR_HZ)
    if Symbol is Dash then
        delay(DASH_TIME_MS)
    else
        delay(DOT_TIME_MS)
    end if
    noTone(TX_PIN)
    delay(INTER_SYMBOL_TIME_MS)
end for
delay(PAUSE_TIME_MS)

```

Damit die Methode `tone()` funktioniert, muss der Pin ein PWM Pin sein, z.B. 3 oder 11. Um die richtigen Symbol Dauern für die Symbole in einem Morse-Codewort zu senden, kann ein Ablauf wie in Algorithmus 1 verwendet werden.

7.1.3 Widerstandsberechnung

Für die IR-Diode soll sich im Betrieb ein Arbeitspunkt von $I = 130 \text{ mA}$ und $U_F = 1.6 \text{ V}$ einstellen. Um den Strom durch die Diode einzustellen, muss der Widerstand R_2 passend dimensioniert werden.

Die restliche Spannung die nicht über die Diode abfällt, muss über R_2 und $R_{DS,on}$ abfallen. Mit $R_{DS,on} = 14 \Omega$ ergibt sich:

$$R_2 = \frac{5 \text{ V} - 1.6 \text{ V}}{130 \text{ mA}} - R_{DS,on} = 12.15 \Omega$$

Um den Maximalstrom nicht zu überschreiten, wird der nächst höhere Widerstand aus der E6 Reihe gewählt.

$$R_2 = 15 \Omega$$

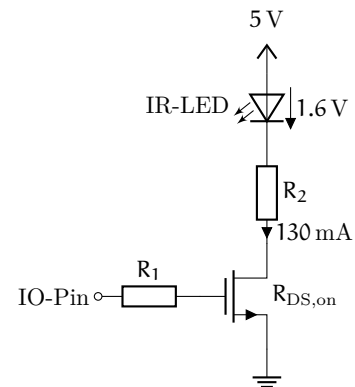


Abbildung 5: NMOS Treiber für die Infrarot LED.

7.2 Durchführung

- Tinker CAD Link
- Stückliste

7.2.1 Aufbau am Steckbrett

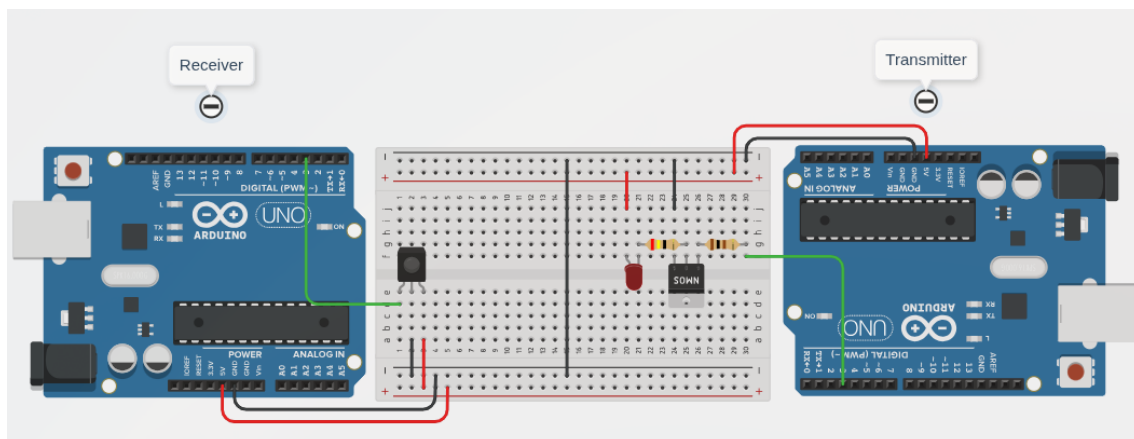


Abbildung 6: Planung des Aufbaus in Tinkercad

Foto vom Aufbau Einfügen

7.2.2 Implementierung des Senders

Um die Buchstaben im Alphabet auf Morsecodewörter umzuwandeln, wird eine Tabelle verwendet, welche die Zusammensetzungen von Dots und Dashes, sowie die Länge des jeweiligen Codewortes speichert. Der Code wird als eine binäre Zahl gespeichert, dabei korrespondiert eine 0 zu einem Dot und eine 1 zu einem Dash.

Listing 1: Globale Variablen und Definitionen des Senders

```

1  #define F_IR_HZ 38000
2
3
4  // Number of UNITS representing a morse symbol
5  typedef enum {
6      UNIT = 1,
7      DOT = 1,
8      DASH = 3,
9      INTER_SYM = 1,
10     PAUSE = 7
11 } morseUnit;
12
13 typedef struct {
14     uint8_t code; // 0 for dot, 1 for dash
15     uint8_t len; // number of syms
16 } morseCode;
17
18 static const morseCode morseTable[26] = {
19     {0b01, 2}, {0b1000, 4}, {0b1010, 4}, {0b100, 3}, // A B C D
20     {0b0, 1}, {0b0010, 4}, {0b110, 3}, {0b0000, 4}, // E F G H
21     {0b00, 2}, {0b0111, 4}, {0b101, 3}, {0b0100, 4}, // I J K L
22     {0b11, 2}, {0b10, 2}, {0b111, 3}, {0b0110, 4}, // M N O P
23     {0b1101, 4}, {0b010, 3}, {0b000, 3}, {0b1, 1}, // Q R S T
24     {0b001, 3}, {0b0001, 4}, {0b011, 3}, {0b1001, 4}, // U V W X
25     {0b1011, 4}, {0b1100, 4} // Y Z
26 };
27
28 const int TX_PIN = 3;
29 static const uint16_t UNIT_LEN_MS = 200;
30 static const uint32_t UNIT_LEN_US = 200000;

```

Listing 2: Methode zum Senden von Charakteren als Morsesymbolen

```

1  static void sendMorseChar(char c) {
2      uint8_t idx = 0;
3      if (c >= 'a' && c <= 'z') {
4          idx = c - 'a';
5      } else if (c >= 'A' && c <= 'Z') {
6          idx = c - 'A';
7      } else {
8          return;
9      }
10
11     morseCode m = morseTable[idx];
12
13     for (int i = m.len - 1; i >= 0; i--) {

```

```

14     tone(TX_PIN, F_IR_HZ);
15     if (m.code & (1 << i)) {
16         delay(DASH * UNIT_LEN_MS);
17     } else {
18         delay(DOT * UNIT_LEN_MS);
19     }
20     noTone();
21     delay(INTER_SYM * UNIT_LEN_MS);
22 }
23 delay(PAUSE * UNIT_LEN_MS);
24 }

```

Listing 3: Setup und Loop des Senders

```

1 void setup() {
2     Serial.begin(9600);
3     pinMode(TX_PIN, OUTPUT);
4 }
5
6 void loop() {
7     // if (Serial.available() > 0) {
8     //     sendMorseChar(Serial.read());
9     // }
10
11     sendMorseChar('Q');
12     sendMorseChar('S');
13     sendMorseChar('L');
14 }

```

7.2.3 Implementierung des Empfängers

Listing 4: Globale Variablen und Definitionen des Empfängers

```

1
2 #define MORSE_TREE_LEN 31
3 #define F_IR_HZ 38000
4
5 // Number of UNITS representing a morse symbol
6 typedef enum {
7     UNIT = 1,
8     DOT = 1,
9     DASH = 3,
10    INTER_SYM = 1,
11    PAUSE = 7
12 } morseUnit;
13
14 static const char morseTree[MORSE_TREE_LEN] = {
15     '\0', // 0
16     'E', 'T', // 1
17     'I', 'A', 'N', 'M', // 2
18     'S', 'U', 'R', 'W', 'D', 'K', 'G', 'O', // 3
19     'H', 'V', 'F', '\0', 'L', '\0', 'P', 'J',
20     'B', 'X', 'C', 'Y', 'Z', 'Q', '\0', '\0' // 4
21 };

```



```
22
23 static const int RX_PIN = 3;
24 static const uint16_t UNIT_LEN_MS = 200;
25 static const uint32_t UNIT_LEN_US = UNIT_LEN_MS * 1000;
26 static const uint32_t TIMEOUT_US = (PAUSE - UNIT) * UNIT_LEN_US;
27 static const uint32_t DOT_THRESH_US = (DOT + UNIT) * UNIT_LEN_US;
28 static const uint32_t DASH_THRESH_US = (DASH + UNIT) * UNIT_LEN_US;
```

Listing 5: Setup und Loop des Empfängers

```
1 void setup() {
2     Serial.begin(9600);
3     pinMode(RX_PIN, INPUT);
4 }
5
6 void loop() {
7     uint8_t morseTreeIdx = 0;
8
9     // IR Sensor yields LOW when receiving
10    uint32_t meas_us = pulseIn(RX_PIN, LOW, TIMEOUT_US);
11
12    while (meas_us && morseTreeIdx < MORSE_TREE_LEN) {
13        // Traverse binary morse tree
14        if (meas_us < DOT_THRESH_US) {
15            morseTreeIdx = 2 * morseTreeIdx + 1;
16        } else if (meas_us < DASH_THRESH_US) {
17            morseTreeIdx = 2 * morseTreeIdx + 2;
18        } else {
19            break;
20        }
21        meas_us = pulseIn(RX_PIN, LOW, TIMEOUT_US);
22    }
23
24    char letter = morseTree[morseTreeIdx];
25    if (letter != '\0') {
26        Serial.println(letter);
27    }
28 }
```

7.3 Ergebnisse